

vtu-easy.com

MODULE – 2

Logic Design with MSI Components and Programmable Logic Devices

Digital System Design Using Verilog (BEC302)
VTU | Department of ECE

Topics Covered

- ✓ Binary Adders & Subtractors
- ✓ Comparators
- ✓ Decoders & Encoders
- ✓ Multiplexers
- ✓ Programmable Logic Devices (PLDs)
- ✓ CPLDs & FPGAs
- ✓ Quick Revision

Prepared exclusively for VTU Students | vtu-easy.com

TABLE OF CONTENTS

1. Design of Combinational Circuits

1.1 Arithmetic Circuits

- 1.1.1 Half Adder
- 1.1.2 Full Adder
- 1.1.3 Half Subtractor
- 1.1.4 Full Subtractor
- 1.1.5 4-Bit Parallel Adder
- 1.1.6 Binary Subtractor
- 1.1.7 4-Bit Binary Subtractor
- 1.1.8 4-Bit Adder/Subtractor
- 1.1.9 Carry Look-Ahead Adder (CLA)

2. Comparators

- 2.1 1-Bit Comparator
- 2.2 2-Bit Comparator
- 2.3 4-Bit Magnitude Comparator (IC 7485)

3. Decoders

- 3.1 2×4 Decoder
- 3.2 3×8 Decoder
- 3.3 BCD to Decimal Decoder
- 3.4 Decoder as Minterm Generator

4. Encoders

- 4.1 Octal-to-Binary (8×3) Encoder
- 4.2 Priority Encoder
- 4.3 BCD Encoder

5. Multiplexers (MUX)

- 5.1 2×1 MUX
- 5.2 4×1 MUX
- 5.3 8×1 MUX
- 5.4 MUX as Universal Logic Element
- 5.5 Demultiplexer (DEMUX)

6. Programmable Logic Devices (PLDs)

- 6.1 ROM
- 6.2 PLA
- 6.3 PAL
- 6.4 GAL

7. Complex PLDs (CPLD)

- 7.1 CPLD Architecture
- 7.2 Features & Applications

8. Field Programmable Gate Arrays (FPGA)

- 8.1 FPGA Architecture
- 8.2 CLB, IOB, Switch Matrix
- 8.3 CPLD vs FPGA Comparison

9. Quick Revision

- 9.1 Formulas & Boolean Expressions
- 9.2 Key Definitions
- 9.3 Important Truth Tables
- 9.4 Important One-Liners

1. Design of Combinational Circuits

A combinational circuit is a digital logic circuit in which the output at any instant of time depends solely on the input values at that instant—there is no memory of past inputs. These circuits are built using logic gates such as AND, OR, NOT, NAND, NOR, XOR, and XNOR. They form the core building blocks of all arithmetic and data processing units inside digital computers, calculators, smartphones, and embedded systems.

Design Procedure

The systematic procedure for designing any combinational circuit follows six well-defined steps:

1. State the problem completely and exactly — identify what arithmetic or logical operation the circuit must perform.
2. Interpret the problem and determine the available input variables and the required output variables.
3. Assign letter symbols to each input and output variable (e.g., A, B, C_{in} for inputs and S, C_{out} for outputs).
4. Construct the truth table that defines the complete required relationship between all input and output combinations.
5. Obtain a simplified Boolean expression for each output using Karnaugh Maps (K-Maps) or Boolean algebra to minimize gate count.
6. Draw the logic circuit diagram implementing the minimized Boolean expressions using logic gates.

1.1 Arithmetic Circuits

Arithmetic circuits perform mathematical operations (addition, subtraction, multiplication, division) in binary form. These circuits are fundamental to ALU (Arithmetic and Logic Unit) design in all processors. Since they operate electronically, typical addition completes in under 1 microsecond.

1.1.1 Half Adder

A Half Adder is the simplest arithmetic circuit that adds two single-bit binary numbers. It has two inputs (A and B) and two outputs: Sum (S) and Carry (C). It is called a 'half' adder because it cannot accept a carry input from a previous stage — limiting its use to the least significant bit (LSB) position only.

Truth Table for Half Adder:

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expressions derived from Truth Table:

$$S = A'B + AB' = A \oplus B \quad (\text{XOR function})$$
$$C = A \cdot B \quad (\text{AND function})$$

The Sum output follows the XOR (Exclusive-OR) pattern — it is 1 only when inputs differ. The Carry output follows the AND pattern — it is 1 only when both inputs are 1.

Logic Gate Implementation:

- One 2-input XOR gate generates the Sum output
- One 2-input AND gate generates the Carry output

Key Limitation of Half Adder

Cannot handle a carry-in from a lower-order bit position.

Therefore, a Half Adder can ONLY be used for the least significant bit (rightmost bit) in a multi-bit binary addition.

For all higher-order bit positions, a Full Adder must be used.

1.1.2 Full Adder

A Full Adder is a combinational circuit that adds THREE single-bit binary numbers: two data bits (A and B) and one carry-in bit (Cin). It produces two outputs: Sum (S) and Carry-out (Cout). A Full Adder is essential for multi-bit binary addition because it can accept carry from a previous less significant stage.

Truth Table for Full Adder:

A	B	Cin	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Expression Derivation for Sum (S):

From the truth table, Sum = 1 for input combinations: A'B'Cin, A'BCin', AB'Cin', ABCin

$$\begin{aligned}
 S &= A'B'Cin + A'BCin' + AB'Cin' + ABCin \\
 &= A' (B'Cin + BCin') + A (B'Cin' + BCin) \\
 &= A' (B \oplus Cin) + A (B \oplus Cin)' \\
 &= A \oplus B \oplus Cin \quad \text{(Triple XOR)}
 \end{aligned}$$

Boolean Expression Derivation for Carry-out (Cout):

From the truth table, Cout = 1 for: A'BCin, AB'Cin, ABCin', ABCin

$$\begin{aligned}
 Cout &= A'BCin + AB'Cin + ABCin' + ABCin \\
 &= BCin + ACin + AB \quad \text{(after K-Map simplification)}
 \end{aligned}$$

Logic Gate Implementation:

- Two 2-input XOR gates for Sum output
- Three 2-input AND gates for partial carry terms
- One 3-input OR gate for final Carry-out

Full Adder using Two Half Adders + OR Gate:

A Full Adder can be economically constructed using two Half Adders and one OR gate:

- First HA: inputs A and B → produces D = A⊕B and carry C1 = AB
- Second HA: inputs D and Cin → produces S = D⊕Cin = A⊕B⊕Cin and carry C2 = D·Cin
- Final Cout = C1 OR C2 = AB + (A⊕B)·Cin

 A Full Adder is the fundamental building block for all multi-bit adders. Cascading n Full Adders forms an n-bit Ripple Carry Adder.

1.1.3 Half Subtractor

A Half Subtractor is a combinational circuit that subtracts one 1-bit binary number (Y, the subtrahend) from another 1-bit binary number (X, the minuend). It has two inputs (X and Y) and two outputs: Difference (D) and Borrow (B). Like the Half Adder, it cannot accept a borrow-in from a previous stage.

Truth Table for Half Subtractor:

X (Minuend)	Y (Subtrahend)	Difference (D)	Borrow (B)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Boolean Expressions:

$$D = X'Y + XY' = X \oplus Y \quad (\text{XOR gate})$$

$$B = X'Y \quad (\text{NOT gate + AND gate})$$

The Difference output is identical in logic to the Sum of a Half Adder (XOR function). The Borrow output is 1 only when X=0 and Y=1 (borrowing from a higher column, i.e., $0 - 1 = 1$ with borrow).

Logic Gate Implementation:

- One XOR gate for Difference output
- One NOT gate on X and one AND gate with X' and Y for Borrow output

1.1.4 Full Subtractor

A Full Subtractor is a combinational circuit used to subtract three single bits: minuend (X), subtrahend (Y), and borrow-in (Z from previous stage). It produces two outputs: Difference (D) and Borrow-out (B). It is the subtraction counterpart of the Full Adder and is essential for multi-bit binary subtraction operations.

Truth Table for Full Subtractor:

X	Y	Z (Bin)	Difference (D)	Borrow (B)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Boolean Expressions:

$$\begin{aligned}
 D &= X \oplus Y \oplus Z \\
 B &= X'Y'Z + X'YZ' + X'YZ + XYZ \\
 &= Z(X \oplus Y) + X'Y
 \end{aligned}$$

Derivation of D:

From minterms where D=1: (0,0,1), (0,1,0), (1,0,0), (1,1,1)

$$D = X'Y'Z + X'YZ' + XY'Z' + XYZ = X \oplus Y \oplus Z$$

Logic Gate Implementation:

- Two XOR gates, two AND gates, one OR gate, and one NOT gate
- Can also be built using two Half Subtractors and an OR gate

 The Full Subtractor's logic for Difference is identical to the Full Adder's Sum (XOR of all three inputs). The difference lies only in the Borrow/Carry logic.

1.1.5 4-Bit Parallel Adder (Ripple Carry Adder)

A 4-Bit Parallel Adder adds two 4-bit binary numbers $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$, producing a 4-bit Sum $S_3S_2S_1S_0$ and a final Carry-out C_4 . It uses four Full Adders connected in cascade (series).

Two Implementation Approaches:

- Approach 1: One Half Adder for LSB (A_0+B_0) and three Full Adders for remaining bits
- Approach 2 (Preferred): Four Full Adders — with $C_0=0$ fed to the rightmost Full Adder, which then behaves like a Half Adder

Operation:

- FA0 adds $A_0, B_0, C_0=0 \rightarrow$ produces S_0 and C_1
- FA1 adds $A_1, B_1, C_1 \rightarrow$ produces S_1 and C_2
- FA2 adds $A_2, B_2, C_2 \rightarrow$ produces S_2 and C_3
- FA3 adds $A_3, B_3, C_3 \rightarrow$ produces S_3 and C_4 (final carry / overflow bit)

Why is it called 'Ripple Carry'?

Each Full Adder must wait for the carry output from the previous stage. The carry 'ripples' from the LSB stage to the MSB stage, causing propagation delay. For an n -bit adder, the worst-case delay = $n \times (\text{delay of one Full Adder})$. This is the major drawback of the ripple carry adder.

Ripple Carry Adder Facts

Maximum output = 4-bit sum + 1 carry = 5 bits total.

Carry-out C_4 from the last Full Adder represents the MSB (overflow indicator).

This design can be extended to any bit-width by cascading more Full Adders.

Also called a Serial Carry Adder due to sequential carry propagation.

1.1.6 Binary Subtractor

A Binary Subtractor performs subtraction of two binary numbers. There are two methods to implement binary subtraction:

Method 1: Cascading Full Subtractors

An n-bit binary subtractor is built by cascading n Full Subtractors. This creates two separate circuits — one for addition and one for subtraction. Each Full Subtractor computes Difference and Borrow, passing the Borrow to the next higher stage.

Method 2: 2's Complement Method (Preferred)

The same binary adder circuit is reused for subtraction by feeding it the 2's complement of the subtrahend B:

$$\begin{aligned} A - B &= A + (\text{2's complement of } B) \\ &= A + (\text{1's complement of } B) + 1 \end{aligned}$$

Steps: (1) Invert all bits of B using NOT gates (1's complement), (2) Add 1 by setting $C_{in}=1$, (3) Feed A and inverted-B into the standard binary adder.

Advantage of 2's Complement Subtraction

No need for a separate subtractor circuit.

The same 4-bit parallel adder handles both addition and subtraction.

Only extra hardware needed: XOR gates controlled by a Mode select line ($M=0$ for add, $M=1$ for subtract).

1.1.7 4-Bit Binary Subtractor

The 4-bit binary subtractor computes $A_3A_2A_1A_0 - B_3B_2B_1B_0$. It uses the same four Full Adders as the 4-bit adder, but with the bits of B complemented and initial carry (borrow) $C_{in} = 1$ to complete the 2's complement addition.

Circuit Description:

- Apply normal bits of A as one operand to the four Full Adders
- Apply the inverted (NOT) bits of B as the second operand
- Set C_0 (initial carry) = 1 to add 1 (completing 2's complement of B)
- The Full Adders then compute $A + (\sim B) + 1 = A - B$

Output Interpretation:

- If $A > B$: MSB (C_4) = 0, remaining 4 bits represent magnitude of (A-B)
- If $A < B$: MSB (C_4) = 1, and the output is the 2's complement of (B-A); take 2's complement of output to get magnitude
- If $A = B$: Output = 0000 with $C_4 = 1$

1.1.8 4-Bit Binary Adder/Subtractor

A 4-bit Binary Adder/Subtractor is a versatile circuit that performs EITHER addition OR subtraction based on a single control signal (Mode, M). This is achieved by adding XOR gates between the B inputs and the Full Adders.

Operation Principle:

- Each bit B_i is passed through an XOR gate: $B_i \oplus M$
- When $M = 0$ (Addition): $B_i \oplus 0 = B_i$ (original B), and $C_0=0 \rightarrow$ Circuit performs $A + B$
- When $M = 1$ (Subtraction): $B_i \oplus 1 = B_i'$ (inverted B), and $C_0=1 \rightarrow$ Circuit performs $A + (\sim B) + 1 = A - B$

$$\text{Output} = A + (B \oplus M) + M$$

XOR Gate as Controlled Inverter

$\text{XOR}(x, 0) = x \rightarrow$ passes x unchanged

$\text{XOR}(x, 1) = x' \rightarrow$ inverts x

This property allows one XOR gate to act as a programmable inverter controlled by M .

 This circuit (Adder/Subtractor) is used inside the Arithmetic Logic Unit (ALU) of all microprocessors.

1.1.9 Carry Look-Ahead Adder (CLA)

The Carry Look-Ahead Adder (CLA) overcomes the speed limitation of the Ripple Carry Adder. Instead of waiting for carry to ripple through each stage, CLA pre-computes all carries simultaneously from the original inputs. This dramatically reduces propagation delay from $O(n)$ to $O(\log n)$.

Two Key Signals per Bit Stage i:

$$\begin{aligned} \text{Generate: } G_i &= A_i \cdot B_i && (\text{carry generated even if } C_{in}=0) \\ \text{Propagate: } P_i &= A_i \oplus B_i && (\text{carry propagated from } C_{in}) \end{aligned}$$

Carry Equations:

$$\begin{aligned} C_1 &= G_0 + P_0 \cdot C_0 \\ C_2 &= G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0 \\ C_3 &= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0 \\ C_4 &= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0 \end{aligned}$$

Sum Equations:

$$S_i = P_i \oplus C_i$$

Parameter	Ripple Carry Adder	Carry Look-Ahead Adder
Speed	Slow — $O(n)$ delay	Fast — $O(\log n)$ delay
Complexity	Simple — less hardware	Complex — more hardware
Carry propagation	Sequential (ripples)	Parallel (simultaneous)
Typical use	Low-speed, low-power	High-speed processors
Gate levels	$2n$ gates	4 gate levels (for 4-bit)

Each bit stage in a CLA contains: a 2-input XOR gate (for P_i and S_i), a 2-input AND gate (for G_i), and the carry computation is done separately in a dedicated look-ahead carry generator block. The CLA reduces the critical path from $2n$ gate delays to just 4 gate delays for a 4-bit addition.

2. Comparators

A Comparator is a combinational circuit that compares two binary numbers and determines their relationship. The three possible outputs are: $A > B$, $A = B$, and $A < B$. Comparators are widely used in control systems, sorting networks, multiplexers, and digital signal processors.

2.1 1-Bit Comparator

Compares two single bits A and B. All three relationships (greater, equal, less) are evaluated using simple gates.

A	B	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

$$\begin{aligned}
 (A > B) &= A \cdot B' \\
 (A = B) &= A \odot B = A'B' + AB \quad (\text{XNOR}) \\
 (A < B) &= A' \cdot B
 \end{aligned}$$

2.2 2-Bit Magnitude Comparator

Compares two 2-bit numbers A_1A_0 and B_1B_0 . The MSB comparison is done first; if MSBs are equal, the LSB comparison decides the result.

Equality Condition:

$$\begin{aligned}
 (A = B) &\text{ iff } A_1=B_1 \text{ AND } A_0=B_0 \\
 E_1 &= A_1 \odot B_1, \quad E_0 = A_0 \odot B_0, \quad \text{Equal} = E_1 \cdot E_0
 \end{aligned}$$

Greater-Than Condition:

$$(A > B) = A_1B_1' + E_1 \cdot A_0B_0'$$

First, check if $A_1 > B_1$. If $A_1=B_1$ ($E_1=1$), then check if $A_0 > B_0$.

2.3 4-Bit Magnitude Comparator — IC 7485

IC 7485 is a standard 4-bit magnitude comparator MSI chip. It compares two 4-bit numbers $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$ and produces three output signals: $(A > B)$, $(A = B)$, $(A < B)$. It also has three cascade inputs for expandability.

Internal Logic (evaluation from MSB to LSB):

- Compare A_3 vs B_3 → if different, result determined immediately
- If $A_3=B_3$, compare A_2 vs B_2
- If $A_2=B_2$, compare A_1 vs B_1
- If $A_1=B_1$, compare A_0 vs B_0
- If all bits equal, cascade inputs determine final output

Cascade Inputs (for multi-chip expansion):

- Connect $(A > B)_{\text{out}}$ of chip N to $(A > B)_{\text{in}}$ of chip N+1

- Two 7485 chips can compare 8-bit numbers
- Four 7485 chips can compare 16-bit numbers

For the equal condition:

$$(A=B) = (A_3 \odot B_3) \cdot (A_2 \odot B_2) \cdot (A_1 \odot B_1) \cdot (A_0 \odot B_0) \cdot (\text{CascadeEQ})$$

 For single chip use, set cascade inputs: $(A > B)_{in}=0$, $(A=B)_{in}=1$, $(A < B)_{in}=0$

3. Decoders

A Decoder is a combinational circuit that converts a binary code at its inputs into a set of individual output lines such that each output corresponds to exactly one minterm of the input variables. An n -to- 2^n decoder has n input lines and up to 2^n output lines (one per minterm). Only ONE output is active (HIGH) at any given time.

Enable Input:

Most practical decoders include an Enable (EN) input. When $EN=0$, all outputs are deactivated regardless of the input code — this feature is used for chip select in memory systems.

3.1 2×4 Decoder (2-to-4 Line Decoder)

A 2×4 decoder has 2 input lines (A_1, A_0) and 4 output lines (D_0, D_1, D_2, D_3). Each output corresponds to one of the four possible input combinations (minterms m_0, m_1, m_2, m_3).

Truth Table:

A1	A0	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Boolean Expressions:

$$\begin{aligned} D_0 &= A_1' \cdot A_0' && \text{(minterm } m_0) \\ D_1 &= A_1' \cdot A_0 && \text{(minterm } m_1) \\ D_2 &= A_1 \cdot A_0' && \text{(minterm } m_2) \\ D_3 &= A_1 \cdot A_0 && \text{(minterm } m_3) \end{aligned}$$

Implementation uses 4 AND gates and 2 NOT gates.

3.2 3×8 Decoder (3-to-8 Line Decoder)

A 3×8 decoder has 3 input lines (A_2, A_1, A_0) and 8 output lines (D_0 to D_7), each representing one of the 8 possible minterms (m_0 to m_7). A popular IC is the 74138.

Truth Table:

A2	A1	A0	Active Output
0	0	0	$D_0 = A_2'A_1'A_0'$
0	0	1	$D_1 = A_2'A_1'A_0$
0	1	0	$D_2 = A_2'A_1A_0'$
0	1	1	$D_3 = A_2'A_1A_0$
1	0	0	$D_4 = A_2A_1'A_0'$
1	0	1	$D_5 = A_2A_1'A_0$
1	1	0	$D_6 = A_2A_1A_0'$
1	1	1	$D_7 = A_2A_1A_0$

Implementation: 8 AND gates (3-input each) and 3 NOT gates.

3.3 BCD-to-Decimal Decoder (4×10)

A BCD-to-Decimal decoder converts a 4-bit BCD (Binary Coded Decimal) code (0000 to 1001) into 10 output lines (0 to 9). Input combinations 1010 to 1111 are invalid/don't care states. The IC 7442 is a standard BCD-to-decimal decoder. Each of the 10 outputs drives a separate indicator (e.g., in a 7-segment display decoder chain, Nixie tubes).

3.4 Decoder as Minterm Generator

A decoder inherently generates all minterms of its input variables. This property is extremely useful for implementing ANY Boolean function using a decoder with OR gates.

Procedure to implement $F(A,B,C)$ using a 3×8 decoder:

7. Identify all minterms (where $F=1$) from the truth table or minterm list
8. Connect the corresponding decoder output lines to an OR gate
9. The OR gate output gives the desired function

Example: Implement $F(A,B,C) = \Sigma m(1,3,5,7)$

$$F = D_1 + D_3 + D_5 + D_7 \quad (\text{using } 3 \times 8 \text{ decoder} + 4\text{-input OR gate})$$

Advantage: Same decoder can implement multiple different functions simultaneously by using separate OR gates for each function.

4. Encoders

An Encoder is a combinational circuit that performs the INVERSE operation of a decoder. It converts one of 2^n input lines into an n -bit binary code at the output. Only ONE input line is active (HIGH) at a time, and the output is the binary code corresponding to that active input. Encoders are used in keyboards, A/D converters, and priority circuits.

4.1 Octal-to-Binary Encoder (8×3)

An 8×3 encoder has 8 input lines (I0 to I7) and 3 output lines (A2, A1, A0). Exactly one input is HIGH at a time, and the 3-bit binary output represents the index of that active input.

Truth Table (active HIGH inputs):

Active Input	A2	A1	A0
I0 (D0)	0	0	0
I1 (D1)	0	0	1
I2 (D2)	0	1	0
I3 (D3)	0	1	1
I4 (D4)	1	0	0
I5 (D5)	1	0	1
I6 (D6)	1	1	0
I7 (D7)	1	1	1

Boolean Expressions:

$$A0 = I1 + I3 + I5 + I7$$

$$A1 = I2 + I3 + I6 + I7$$

$$A2 = I4 + I5 + I6 + I7$$

Implementation uses only 3 OR gates (no AND/NOT gates needed).

Limitation:

A simple encoder cannot handle the case when multiple inputs are HIGH simultaneously, and it produces an incorrect output if I0 is active (output = 000, indistinguishable from no-input case).

4.2 Priority Encoder

A Priority Encoder is an improved encoder that resolves the problem of multiple simultaneous active inputs. It assigns a PRIORITY to each input line. When multiple inputs are HIGH simultaneously, the output corresponds to the input with the HIGHEST priority (usually the highest-numbered active input).

4-to-2 Priority Encoder Truth Table (I3 has highest priority):

I3	I2	I1	I0	A1	A0	V (Valid)
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1

I3	I2	I1	I0	A1	A0	V (Valid)
1	X	X	X	1	1	1

V (Valid) output = 1 when at least one input is active; V=0 when no input is active.

Boolean Expressions:

$$\begin{aligned}A0 &= I3 + I1 \cdot I2' \\A1 &= I3 + I2 \\V &= I0 + I1 + I2 + I3\end{aligned}$$

 The 'X' in the truth table represents a don't-care condition — the output is determined by the highest-priority active input regardless of lower-priority inputs.

4.3 BCD Encoder

A BCD Encoder converts a decimal digit (0–9) represented by 10 individual input lines into its 4-bit BCD (8421) code. It has 10 inputs (one per decimal digit) and 4 outputs (BCD bits D, C, B, A). The IC 74147 is a standard 10-line to 4-line priority encoder.

$$\begin{aligned}B3 &= I8 + I9 \\B2 &= I4 + I5 + I6 + I7 \\B1 &= I2 + I3 + I6 + I7 \\B0 &= I1 + I3 + I5 + I7 + I9\end{aligned}$$

5. Multiplexers (MUX)

A Multiplexer (MUX) or Data Selector is a combinational circuit that selects ONE of several (2n) input data lines and routes it to a single output line under the control of n SELECT (address) lines. Think of it as a digitally-controlled rotary switch. MUXes are among the most versatile MSI components.

5.1 2×1 Multiplexer

The simplest MUX with 2 data inputs (I0, I1), 1 select line (S), and 1 output (Y).

S	Output Y
0	I0 (selects input 0)
1	I1 (selects input 1)

$$Y = S' \cdot I0 + S \cdot I1$$

Implementation: 2 AND gates + 1 OR gate + 1 NOT gate.

5.2 4×1 Multiplexer

A 4×1 MUX has 4 data inputs (I0–I3), 2 select lines (S1, S0), and 1 output (Y). The IC 74153 is a dual 4×1 MUX.

S1	S0	Output Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

$$Y = S1'S0' \cdot I0 + S1'S0 \cdot I1 + S1S0' \cdot I2 + S1S0 \cdot I3$$

Implementation: 4 AND gates (3-input each) + 1 OR gate (4-input) + 2 NOT gates.

5.3 8×1 Multiplexer

An 8×1 MUX has 8 data inputs (I0–I7), 3 select lines (S2, S1, S0), and 1 output (Y). The IC 74151 is a standard 8×1 MUX used widely in digital systems.

$$Y = \sum (Si'Sj'Sk' \cdot Ii) \quad \text{for all 8 input combinations}$$

The select lines S2S1S0 form a 3-bit binary address. For example, S2S1S0 = 101 routes I5 to the output.

5.4 MUX as Universal Logic Element

A Multiplexer can implement ANY Boolean function using only the MUX itself — no additional gates needed. This is one of the most powerful features of a MUX.

Method 1: Using an n-variable function with a 2n×1 MUX:

10. Write the truth table of the function
11. Connect the n input variables to the n select lines

12. For each row of the truth table, feed the output value (0 or 1) as the corresponding MUX data input

Example: A 3-variable function needs an 8×1 MUX. Connect A,B,C to S2,S1,S0 and feed truth table output values to I0–I7.

Method 2: Using one variable fewer (Shannon's Expansion):

13. Use a $2^{(n-1)} \times 1$ MUX for an n-variable function
14. Connect (n-1) variables to select lines
15. Feed the nth variable (X_n) or its complement or 0/1 as data inputs

Example: Use a 4×1 MUX for a 3-variable function. Connect A,B to S1,S0 and feed C, C', 0, or 1 to I0–I3 based on function values.

MUX as Universal Logic Element — Why it works

A 2^n -to-1 MUX can select any one of 2^{2^n} possible functions of n variables.

This is because each data input I_i controls one minterm.

Setting data inputs to 0 or 1 enables or disables specific minterms.

Any Boolean function is a sum of minterms $\rightarrow$ any function can be realized.

5.5 Demultiplexer (DEMUX)

A Demultiplexer (DEMUX) performs the INVERSE operation of a MUX. It takes a SINGLE input data line and distributes it to ONE of 2^n output lines based on n select lines. A 1-to-4 DEMUX (IC 74155) is commonly used.

$$1 \times 4 \text{ DEMUX: } Y_0 = I \cdot S_1' S_0', \quad Y_1 = I \cdot S_1' S_0, \quad Y_2 = I \cdot S_1 S_0', \quad Y_3 = I \cdot S_1 S_0$$

Applications: Data routing in communication systems, address decoding, memory chip select logic.



A Decoder with Enable input can function as a DEMUX: the Enable input becomes the data input, and the address inputs act as select lines.

6. Programmable Logic Devices (PLDs)

Programmable Logic Devices (PLDs) are integrated circuits that can be configured by the designer (after manufacturing) to implement any desired Boolean function. Unlike fixed-function MSI chips, PLDs offer flexibility without requiring custom IC fabrication. They are based on arrays of AND and OR gates with programmable connections.

General PLD Structure:

All PLDs consist of two main planes:

- AND Plane (Product Term Generator): generates AND terms (product terms) from the input variables and their complements
- OR Plane (Sum of Products): ORs selected product terms to form the output functions

Types of PLDs based on programmability:

Type	AND Plane	OR Plane	Notes
ROM	Fixed (decoder)	Programmable	All minterms generated
PLA	Programmable	Programmable	Most flexible, but complex
PAL	Programmable	Fixed	Most popular, fast
GAL	Programmable	Fixed+OLMCs	Re-programmable PAL

6.1 Read Only Memory (ROM)

A ROM is a PLD where the AND array is a fixed complete decoder (generates all 2^n minterms), and the OR array is programmable. Since all minterms are always available, any Boolean function can be implemented by simply programming which minterms to OR together.

ROM as a PLD:

- n-input ROM with m outputs implements m Boolean functions of n variables
- AND array: a complete n-to- 2^n decoder — generates all 2^n minterms
- OR array: programmable connections — each output ORs selected minterms
- Storage capacity: $2^n \times m$ bits (one bit per output per minterm)

Example: 2-input ROM (4 minterms) with 2 outputs:

$$F1 = m_0 + m_2 + m_3 = A'B' + AB' + AB$$

$$F2 = m_1 + m_3 = A'B + AB$$

Program the OR connections at positions (F1,m0), (F1,m2), (F1,m3), (F2,m1), (F2,m3).

6.2 Programmable Logic Array (PLA)

A PLA has BOTH the AND plane and the OR plane programmable. This is the most flexible PLD but also the most expensive and complex to program. Because both planes are programmable, a PLA does not need to generate all 2^n minterms — it can generate only the required product terms, saving silicon area.

PLA Structure:

- AND array: p programmable AND gates ($p < 2n$), each can accept any subset of input variables or complements
- OR array: m programmable OR gates, each sums selected AND terms
- A PLA with n inputs, p product terms, and m outputs is denoted: $n \times p \times m$ PLA

Advantages of PLA:

- More efficient than ROM for functions with many don't-cares
- Minimized expressions (not canonical minterms) can be used → fewer AND gates
- Both planes programmable → maximum flexibility

Disadvantages:

- Slower than PAL due to double programmable array delays
- More complex programming and verification

6.3 Programmable Array Logic (PAL)

A PAL is a PLD where the AND plane is programmable but the OR plane is FIXED. The fixed OR connections determine how many product terms each output can sum. This trade-off (less flexibility than PLA) results in faster operation and simpler programming. PAL is the most widely used PLD type.

PAL Architecture:

- n input lines and their complements available to a programmable AND array
- Each output has a fixed number of AND terms ORed together (typically 4–8)
- Output can often be fed back as an input (for sequential logic)
- Some PALs include registered outputs (with flip-flops) for sequential circuits

Common PAL devices:

- PAL16L8: 16 inputs, 8 active-low outputs
- PAL16R8: 16 inputs, 8 registered outputs (flip-flops)
- PAL22V10: 22 inputs, 10 variable outputs with OLMC (Output Logic Macro Cell)

6.4 Generic Array Logic (GAL)

A GAL is an electrically erasable and reprogrammable version of a PAL. GALs use EEPROM (Electrically Erasable PROM) technology, allowing them to be erased and reprogrammed multiple times (typically 100+ times). The key feature is the Output Logic Macro Cell (OLMC).

Output Logic Macro Cell (OLMC):

Each output of a GAL contains an OLMC that can be configured to act as:

- Combinational active-high output
 - Combinational active-low output
 - Registered output (with D flip-flop)
 - Tri-state output
-
- Most popular: GAL16V8 and GAL22V10
 - Re-programmability makes them ideal for prototyping

7. Complex Programmable Logic Devices (CPLDs)

As digital designs grew larger, simple PLDs became insufficient for implementing complex circuits with 20+ Boolean functions and hundreds of logic gates. CPLDs were developed to overcome this scalability limitation. A CPLD integrates multiple PLD blocks connected by a programmable switch matrix on a single chip.

7.1 CPLD Architecture

A CPLD consists of three main components:

1. PLD Blocks (Function Blocks / Logic Array Blocks):

Multiple independent PLD-like blocks, each capable of implementing several Boolean functions. Each block contains AND/OR arrays similar to a PAL, plus macrocells with flip-flops for registered logic.

2. Programmable Switch Matrix (PSM):

A central routing fabric that interconnects all PLD blocks and I/O blocks. Each PLD block's inputs can come from any I/O pin or any other PLD block's output through the PSM. This provides global connectivity with predictable timing.

3. I/O Blocks:

Bidirectional I/O cells at the chip boundary. Each I/O cell can be configured as input, output, or bidirectional with programmable slew rate, pull-up, and drive strength.

Typical CPLD Block Diagram (connectivity):

I/O Block ↔ PSM ↔ PLD Block 1



PLD Block 2, 3, 4 ... n



I/O Block

CPLD Internal Signal Flow:

16. Input arrives at I/O block
17. Routed through PSM to appropriate PLD block
18. PLD block implements the Boolean logic
19. Output routes back through PSM to I/O block
20. Final signal driven to output pin

7.2 Features and Applications of CPLD

Key Features:

- Instant-on: CPLDs begin operating immediately at power-up (non-volatile configuration storage using EEPROM/Flash)
- Non-volatile: Retain programming after power-down — no need to reload configuration
- Deterministic timing: Simpler routing structure allows predictable, consistent signal delays
- Low idle power: Modern CPLDs (e.g., Xilinx CoolRunner-II) consume as little as 50 μ A idle
- Moderate density: Typically 32 to 512 macrocells — suited for glue logic and control circuits
- Security: Built-in security bit prevents reading back the programmed configuration

- In-System Programmability (ISP): Can be programmed while mounted on PCB via JTAG interface

Applications:

- Bus interface and address decoding logic (when I/O pins > 32)
- State machines and control sequencers
- Television, set-top box, and industrial automation circuits
- Glue logic between different bus standards (PCI, I2C, SPI)
- Power management and sequencing circuits
- Implementation of large combinational circuits



A CPLD can implement more than 20 Boolean functions efficiently. For designs requiring thousands of logic functions, an FPGA is more appropriate.

8. Field Programmable Gate Arrays (FPGAs)

An FPGA is an integrated circuit containing a massive array of configurable logic blocks (CLBs) connected by a programmable interconnect network. Unlike CPLDs with fixed AND/OR arrays, FPGAs use Look-Up Tables (LUTs) as the fundamental logic element — enabling them to implement any logic function of a small number of inputs. FPGAs can be reprogrammed in the field by the user (hence 'Field Programmable'), and they are used for everything from prototyping ASICs to implementing complete microprocessors.

Key Concept — Look-Up Table (LUT):

A k-input LUT is essentially a small SRAM truth table with 2^k memory cells. By writing different values into these cells, the LUT can implement any Boolean function of k inputs. A 4-input LUT has $2^4=16$ cells, capable of representing any function of 4 variables.

8.1 FPGA Architecture

An FPGA chip is organized as a 2D array of configurable logic blocks surrounded by I/O blocks at the boundary, with programmable interconnects (routing channels) running between all blocks.

Three Main Structural Components:

21. Configurable Logic Blocks (CLBs) — core logic cells
22. Input/Output Blocks (IOBs) — interface with external world
23. Programmable Interconnects / Switch Matrix — routing between CLBs and IOBs

Configuration Storage:

Most FPGAs use SRAM-based configuration cells. The entire logic and routing configuration is stored in SRAM bits. When power is applied, the FPGA must load the configuration bitstream from an external Flash/EEPROM before it can operate. This loading takes tens to hundreds of milliseconds.

8.2 CLB, IOB, and Switch Matrix

Configurable Logic Block (CLB) — detailed:

Each CLB is the fundamental logic building block of an FPGA. A typical CLB contains:

- Look-Up Tables (LUTs): Typically 4-input or 6-input LUTs to implement combinational logic functions
- Flip-Flops / Registers: D-type flip-flops for sequential logic (registers, counters, state machines)
- Carry Logic: Dedicated fast carry chain for efficient arithmetic operations
- Multiplexers: Internal programmable MUXes for flexible signal routing within the CLB
- An 8-bit function generator and two 16-bit function generators (in Xilinx CLBs)

Each CLB has inputs on all sides, making them flexible for mapping and partitioning large logic circuits.

Input/Output Blocks (IOB) / I/O Pads:

IOBs are located at the periphery of the FPGA chip and interface between internal logic and external pins. Each IOB supports:

- Input path with optional flip-flop and input delay element
- Output path with tri-state control and slew rate selection

- Bidirectional data flow
- Support for multiple I/O standards: LVCMOS, LVDS, SSTL, etc.
- Pull-up, pull-down, and keeper options

Switch Matrix / Programmable Interconnect:

The Switch Matrix connects the long and short routing channels in a flexible, programmable way. It contains programmable pass transistors (SRAM-controlled transmission gates) that can create connections between different wires. The interconnect includes:

- Long lines: Span the entire chip for global signals (clock, reset, enable)
- Short lines: Connect nearby CLBs for local routing
- General routing matrix: 6-input programmable switch boxes at intersections

Additional Hard-IP Blocks in Modern FPGAs:

- Block RAM (BRAM): Dedicated on-chip SRAM blocks (typically 18Kb or 36Kb each)
- DSP Blocks: Dedicated multiplier-accumulator units for signal processing
- PLL / DCM / MMCM: Clock management blocks for clock synthesis and phase adjustment
- PCIe / Ethernet / USB hard cores: Pre-built high-speed interfaces
- Embedded processors: ARM Cortex-A9 cores in Xilinx Zynq-7000 series

8.3 CPLD vs FPGA — Detailed Comparison

Feature	CPLD	FPGA
Start-up	Instant-on (powers up ready)	Requires config load from ROM (~ms delay)
Configuration storage	Non-volatile (Flash/EEPROM)	Volatile (SRAM — lost at power-off)
Timing	Deterministic, predictable	Complex, tool-assisted timing closure
Logic density	Small (32–512 macrocells)	Massive (thousands–millions of LUTs)
Idle power	Very low (~50 μ A typical)	Higher (SRAM leakage)
Cost	Cheaper for simple designs	More expensive but far more capable
Security	High — internal non-volatile	Risk of bitstream exposure; encryption available
Flexibility	Moderate — AND/OR based	Maximum — LUT-based, any function
Hard IPs	None	BRAM, DSP, PLL, PCIe, Ethernet, CPU
Re-config	Requires power-cycle	Partial reconfiguration while running
Best use	Control logic, state machines	Complex systems, prototyping ASICs
Examples	Xilinx XC9500, Altera MAX	Xilinx Artix-7, Intel Cyclone V

When to choose CPLD vs FPGA

Choose CPLD: Simple glue logic, bus interfaces, I/O expansion, instant-on needed, low power battery designs.

Choose FPGA: Complex digital systems, DSP, image processing, high-speed protocol implementation, ASIC prototyping, or when you need BRAM/DSP/PLL resources.

Rule of thumb: If you need < 512 macrocells → CPLD; if you need more → FPGA.

FPGA Programming Flow:

24. Design Entry: HDL (Verilog/VHDL) or schematic capture
25. Synthesis: Convert HDL to netlist of primitives (LUTs, FFs)
26. Implementation: Place and Route — assign cells to physical CLB locations and route connections
27. Bitstream Generation: Create binary configuration file (.bit file)
28. Configuration: Download bitstream to FPGA via JTAG or from external Flash

9. Quick Revision — VTU Exam Ready

All essential formulas, definitions, tables, and one-liners for last-minute revision | vtU-easy.com

9.1 All Boolean Expressions (Formula Sheet)

ADDERS & SUBTRACTORS

Half Adder: $S = A \oplus B$ | $C = A \cdot B$

Full Adder: $S = A \oplus B \oplus C_{in}$ | $C_{out} = BC_{in} + AC_{in} + AB$

Half Subtractor: $D = X \oplus Y$ | $B = X'Y$

Full Subtractor: $D = X \oplus Y \oplus Z$ | $B = Z(X \oplus Y) + X'Y$

4-Bit Subtractor: $A - B = A + (\sim B) + 1$ (2's complement)

CARRY LOOK-AHEAD ADDER

Generate: $G_i = A_i \cdot B_i$ Propagate: $P_i = A_i \oplus B_i$

$C_1 = G_0 + P_0 \cdot C_0$

$C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$

$C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$

$C_4 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + \dots + P_3 P_2 P_1 P_0 \cdot C_0$

DECODER OUTPUTS (3×8)

$D_0 = A_2'A_1'A_0'$ $D_1 = A_2'A_1'A_0$ $D_2 = A_2'A_1A_0'$ $D_3 = A_2'A_1A_0$

$D_4 = A_2A_1'A_0'$ $D_5 = A_2A_1'A_0$ $D_6 = A_2A_1A_0'$ $D_7 = A_2A_1A_0$

ENCODER OUTPUTS (8×3)

$A_0 = I_1 + I_3 + I_5 + I_7$ $A_1 = I_2 + I_3 + I_6 + I_7$ $A_2 = I_4 + I_5 + I_6 + I_7$

4×1 MUX OUTPUT

$Y = S_1'S_0' \cdot I_0 + S_1'S_0 \cdot I_1 + S_1S_0' \cdot I_2 + S_1S_0 \cdot I_3$

COMPARATOR (1-bit)

$A > B = A \cdot B'$

$A = B = A \odot B = A'B' + AB$

$A < B = A' \cdot B$

9.2 Key Definitions (2-mark answers)

Term	Definition
Half Adder	Adds 2 one-bit numbers; outputs Sum and Carry; cannot accept Carry-in
Full Adder	Adds 3 bits (A, B, Cin); outputs Sum and Carry-out; builds multi-bit adders
Half Subtractor	Subtracts 2 bits; outputs Difference and Borrow; no Borrow-in
Full Subtractor	Subtracts 3 bits (X, Y, Z); outputs Difference and Borrow-out
Ripple Carry Adder	4 cascaded Full Adders; carry ripples from LSB to MSB; slow but simple
CLA Adder	Pre-computes all carries simultaneously using Generate and Propagate signals; fast
Comparator	Circuit comparing two binary numbers; outputs $A > B$, $A = B$, $A < B$
Decoder	n inputs $\rightarrow 2^n$ outputs; only one output HIGH at a time; generates minterms
Encoder	2^n inputs $\rightarrow n$ outputs; inverse of decoder; one input HIGH at a time
Priority Encoder	Multiple inputs may be HIGH; highest-priority input determines output
Multiplexer	2^n inputs + n select lines $\rightarrow 1$ output; digital switch; data selector
Demultiplexer	1 input $\rightarrow 2^n$ outputs based on n selects; inverse of MUX
ROM	Fixed AND array (decoder) + programmable OR; implements truth table
PLA	Both AND and OR arrays programmable; most flexible PLD
PAL	Programmable AND + fixed OR; fast; most commonly used PLD
GAL	EEPROM-based re-programmable PAL with OLMC outputs
CPLD	Multiple PLD blocks + switch matrix; non-volatile; instant-on
FPGA	Array of CLBs (LUTs+FFs) + programmable interconnect; SRAM-based; reconfigurable
LUT	Look-Up Table: small SRAM used as a truth table to implement any logic function
CLB	Configurable Logic Block: basic FPGA cell containing LUTs, FFs, carry logic
OLMC	Output Logic Macro Cell in GAL — configurable as combinational or registered output

9.3 Important Truth Tables at a Glance

Half Adder:

A	B	S ($A \oplus B$)	C (AB)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half Subtractor:

X	Y	D ($X \oplus Y$)	B ($X'Y$)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Full Adder (condensed):

A	B	Cin	S	Cout
0	0	0	0	0
0	1	1	1	0
1	0	1	1	0
1	1	0	0	1
1	1	1	1	1

2×4 Decoder:

A1	A0	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

4×1 MUX:

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

1-Bit Comparator:

A	B	A>B	A=B	A<B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

9.4 Important One-Liners for VTU Exams

Remember These!

- Half Adder = XOR (Sum) + AND (Carry). No Carry-in.
- Full Adder Sum = $A \oplus B \oplus C_{in}$. Carry = $AB + BC_{in} + AC_{in}$.
- A Full Adder = 2 Half Adders + 1 OR gate.
- Ripple Carry is slow because carry propagates sequentially.
- CLA reduces delay to $O(\log n)$ using Generate ($G_i = A_i B_i$) and Propagate ($P_i = A_i \oplus B_i$).
- Decoder generates ALL minterms. Use OR gate to implement any Boolean function.
- Encoder = inverse of decoder. Priority encoder resolves multiple active inputs.
- MUX can implement ANY Boolean function — universal logic element.
- ROM: fixed AND, programmable OR. PLA: both programmable. PAL: prog AND, fixed OR.
- CPLD: non-volatile, instant-on, deterministic timing, low power, AND/OR based.
- FPGA: SRAM-based, needs config load at startup, LUT-based, massive density, supports BRAM/DSP.
- CPLD is better for glue logic. FPGA is better for complex systems and ASIC prototyping.

IC Numbers to Remember

IC 7482 / 74283 — 4-Bit Full Adder
IC 7485 — 4-Bit Magnitude Comparator
IC 74138 — 3×8 Decoder (with enable)
IC 7442 — BCD-to-Decimal Decoder
IC 74151 — 8×1 Multiplexer
IC 74153 — Dual 4×1 Multiplexer
IC 74147 — 10-line to 4-line Priority Encoder
GAL16V8/22V10 — Generic Array Logic (re-programmable)

Best of Luck for your VTU Exam! | vtu-easy.com

This document was prepared for VTU BEC302 students. All rights reserved.